

Egzaminas_word

Generated by Doxygen 1.17.0

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 UrlFinder Class Reference	5
3.1.1 Detailed Description	5
3.1.2 Member Function Documentation	5
3.1.2.1 arURL()	5
3.1.2.2 isvedimas()	6
3.1.2.3 nuskaitymas()	6
3.1.2.4 sutvarkytiURL()	7
3.2 Word Class Reference	7
3.2.1 Detailed Description	8
3.2.2 Constructor & Destructor Documentation	8
3.2.2.1 Word() [1/5]	8
3.2.2.2 Word() [2/5]	8
3.2.2.3 Word() [3/5]	9
3.2.2.4 Word() [4/5]	9
3.2.2.5 Word() [5/5]	9
3.2.2.6 ~Word()	10
3.2.3 Member Function Documentation	10
3.2.3.1 didintiCount()	10
3.2.3.2 getCount()	10
3.2.3.3 getSakiniai()	10
3.2.3.4 getWord()	11
3.2.3.5 operator=() [1/2]	11
3.2.3.6 operator=() [2/2]	11
3.2.3.7 pridetiSakini()	12
3.2.3.8 setCount()	12
3.2.3.9 setWord()	12
3.2.4 Friends And Related Symbol Documentation	12
3.2.4.1 operator<<	12
3.3 WordCounter Class Reference	13
3.3.1 Detailed Description	13
3.3.2 Member Function Documentation	13
3.3.2.1 isvedimas()	13
3.3.2.2 nuskaitymas()	14
4 File Documentation	15
4.1 bibliotekos.h File Reference	15

4.2 bibliotekos.h	16
4.3 main.cpp File Reference	16
4.3.1 Function Documentation	17
4.3.1.1 main()	17
4.4 main.cpp	18
4.5 urlfinder.cpp File Reference	18
4.6 urlfinder.cpp	18
4.7 urlfinder.h File Reference	20
4.8 urlfinder.h	20
4.9 word.cpp File Reference	21
4.9.1 Function Documentation	21
4.9.1.1 operator<<()	21
4.10 word.cpp	21
4.11 word.h File Reference	22
4.12 word.h	23
4.13 wordcounter.cpp File Reference	24
4.14 wordcounter.cpp	24
4.15 wordcounter.h File Reference	25
4.16 wordcounter.h	26
Index	27

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

UrlFinder	5
Word	7
WordCounter	13

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

bibliotekos.h	15
main.cpp	16
urlfinder.cpp	18
urlfinder.h	20
word.cpp	21
word.h	22
wordcounter.cpp	24
wordcounter.h	25

Chapter 3

Class Documentation

3.1 UrlFinder Class Reference

```
#include <urlfinder.h>
```

Public Member Functions

- bool [arURL](#) (const string &zodis)
funkcija, kuri patikrina ar zodis yra URL adresas
- string [sutvarkytiURL](#) (const string &zodis)
funkcija, kuri pašalina galimus skyrybos ženklus iš URL pradžios ir pabaigos
- void [nuskaitymas](#) (const string &failas)
funkcija, kuri skaito failą ir ieško URL adresu
- void [isvedimas](#) (const string &failas)

3.1.1 Detailed Description

Definition at line 4 of file [urlfinder.h](#).

3.1.2 Member Function Documentation

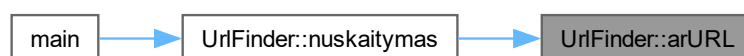
3.1.2.1 arURL()

```
bool UrlFinder::arURL (  
    const string & zodis)
```

funkcija, kuri patikrina ar zodis yra URL adresas

Definition at line 3 of file [urlfinder.cpp](#).

Here is the caller graph for this function:



3.1.2.2 isvedimas()

```
void UrlFinder::isvedimas (  
    const string & failas)
```

Definition at line 67 of file [urlfinder.cpp](#).

Here is the caller graph for this function:



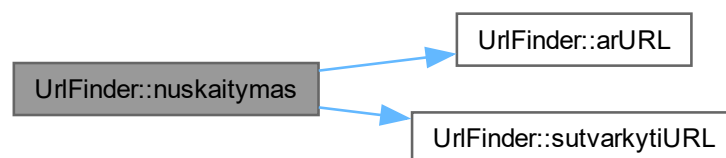
3.1.2.3 nuskaitymas()

```
void UrlFinder::nuskaitymas (  
    const string & failas)
```

funkcija, kuri skaito faila ir iesko URL adresu

Definition at line 49 of file [urlfinder.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.2.4 sutvarkytiURL()

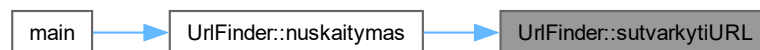
```
string UrlFinder::sutvarkytiURL (
    const string & zodis)
```

funkcija, kuri pašalina galimus skyrybos ženklus iš URL pradžios ir pabaigos

Pašaliname galimus skyrybos ženklus iš URL pradžios ir pabaigos

Definition at line 13 of file [urlfinder.cpp](#).

Here is the caller graph for this function:



The documentation for this class was generated from the following files:

- [urlfinder.h](#)
- [urlfinder.cpp](#)

3.2 Word Class Reference

```
#include <word.h>
```

Public Member Functions

- [Word](#) ()
defaultinis konstruktorius
- [Word](#) (const string &word, int count)
konstruktorius su žodžiu ir skaičiumi
- [Word](#) (const string &word, int count, int sakinyss)
konstruktorius su žodžiu, skaičiumi ir sakinio numeriu
- [Word](#) (const [Word](#) &zodis)
kopijavimo konstruktorius
- [Word](#) & [operator=](#) (const [Word](#) &zodis)
kopijavimo priskyrimo operatorius
- [Word](#) ([Word](#) &&zodis)
move konstruktorius
- [Word](#) & [operator=](#) ([Word](#) &&zodis)
move priskyrimo operatorius
- [~Word](#) ()
destruktorius
- const string & [getWord](#) () const
- int [getCount](#) () const
- void [setWord](#) (const string &word)
- void [setCount](#) (int count)
- void [didintiCount](#) ()
- void [pridetiSakini](#) (int sakinioNumeris)
- const std::set< int > & [getSakiniai](#) () const

Friends

- `std::ostream & operator<< (std::ostream &os, const Word &zodis)`
perdengtas išvedimo operatorius

3.2.1 Detailed Description

Definition at line 5 of file [word.h](#).

3.2.2 Constructor & Destructor Documentation

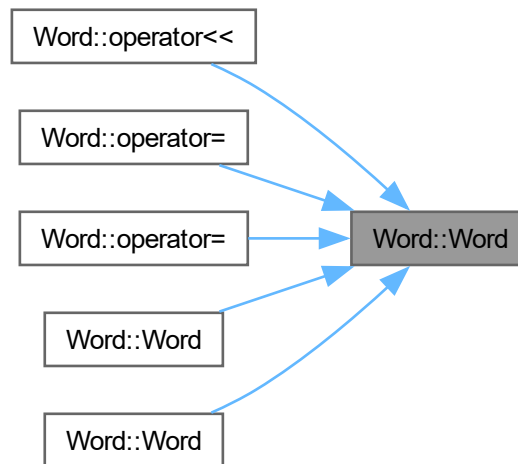
3.2.2.1 Word() [1/5]

```
Word::Word ()
```

defaultinis konstruktorius

Definition at line 4 of file [word.cpp](#).

Here is the caller graph for this function:



3.2.2.2 Word() [2/5]

```
Word::Word (  
    const string & word,  
    int count)
```

konstruktorius su žodžiu ir skaičiumi

Definition at line 10 of file [word.cpp](#).

3.2.2.3 Word() [3/5]

```
Word::Word (  
    const string & word,  
    int count,  
    int sakinyys)
```

konstruktorius su žodžiu, skaičiumi ir sakinio numeriu

Definition at line 15 of file [word.cpp](#).

3.2.2.4 Word() [4/5]

```
Word::Word (  
    const Word & zodis)
```

kopijavimo konstruktorius

Definition at line 21 of file [word.cpp](#).

Here is the call graph for this function:



3.2.2.5 Word() [5/5]

```
Word::Word (  
    Word && zodis)
```

move konstruktorius

Definition at line 37 of file [word.cpp](#).

Here is the call graph for this function:



3.2.2.6 ~Word()

```
Word::~~Word ()
```

destruktorius

Definition at line 59 of file [word.cpp](#).

3.2.3 Member Function Documentation

3.2.3.1 didintiCount()

```
void Word::didintiCount () [inline]
```

Definition at line 25 of file [word.h](#).

3.2.3.2 getCount()

```
int Word::getCount () const [inline]
```

Definition at line 21 of file [word.h](#).

Here is the caller graph for this function:



3.2.3.3 getSakiniai()

```
const std::set< int > & Word::getSakiniai () const [inline]
```

Definition at line 27 of file [word.h](#).

Here is the caller graph for this function:



3.2.3.4 `getWord()`

```
const string & Word::getWord () const [inline]
```

Definition at line 20 of file [word.h](#).

Here is the caller graph for this function:



3.2.3.5 `operator=()` [1/2]

```
Word & Word::operator= (  
    const Word & zodis)
```

kopijavimo priskyrimo operatorius

Definition at line 27 of file [word.cpp](#).

Here is the call graph for this function:



3.2.3.6 `operator=()` [2/2]

```
Word & Word::operator= (  
    Word && zodis)
```

move priskyrimo operatorius

Definition at line 46 of file [word.cpp](#).

Here is the call graph for this function:



3.2.3.7 pridetisakini()

```
void Word::pridetisakini (
    int sakinioNumeris) [inline]
```

Definition at line 26 of file [word.h](#).

Here is the caller graph for this function:



3.2.3.8 setCount()

```
void Word::setCount (
    int count) [inline]
```

Definition at line 23 of file [word.h](#).

3.2.3.9 setWord()

```
void Word::setWord (
    const string & word) [inline]
```

Definition at line 22 of file [word.h](#).

3.2.4 Friends And Related Symbol Documentation

3.2.4.1 operator<<

```
std::ostream & operator<< (
    std::ostream & os,
    const Word & zodis) [friend]
```

perdengtas išvedimo operatorius

Definition at line 65 of file [word.cpp](#).

The documentation for this class was generated from the following files:

- [word.h](#)
- [word.cpp](#)

3.3 WordCounter Class Reference

```
#include <wordcounter.h>
```

Public Member Functions

- void [nuskaitymas](#) (const string &failas)
funkcija, kuri skaito faila ir skaiciuoja zodzius, ju kieki ir sakinius, kuriuose jie yra
- void [isvedimas](#) (const string &failas)
funkcija, kuri isveda zodzius, ju kieki ir sakinius, kuriuose jie yra i faila

3.3.1 Detailed Description

Definition at line 4 of file [wordcounter.h](#).

3.3.2 Member Function Documentation

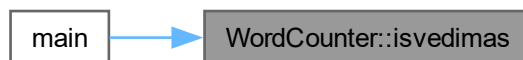
3.3.2.1 isvedimas()

```
void WordCounter::isvedimas (  
    const string & failas)
```

funkcija, kuri isveda zodzius, ju kieki ir sakinius, kuriuose jie yra i faila

Definition at line 81 of file [wordcounter.cpp](#).

Here is the caller graph for this function:



3.3.2.2 nuskaitymas()

```
void WordCounter::nuskaitymas (
    const string & failas)
```

funkcija, kuri skaito failą ir skaičiuoja žodžius, jų kieki ir sakinius, kuriuose jie yra

tikriname ar eilutė baigiasi sakinio pabaigos ženklu, jei taip, tai kitame žodyje padidinsime sakinio numerį

npos - tai reikšmė, kurią grąžina find funkcija, jei neranda ieškomo simbolio. Jei simbolis randamas, tai grąžina jo poziciją eilutėje, o jei nerandamas, tai grąžina npos.

patikriname ar eilutė yra žodis, jei ne, tai tuscia bus tuscia string, o jei taip, tai tuscia bus žodis be skyrybos ženklu ir mažosios raidės

pridedama į žodžių žemėlapi žodžio reikšmę ir count 1. žodžio reikšmę

jei žodis jau yra, tai padidiname count reikšmę

Definition at line 45 of file [wordcounter.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



The documentation for this class was generated from the following files:

- [wordcounter.h](#)
- [wordcounter.cpp](#)

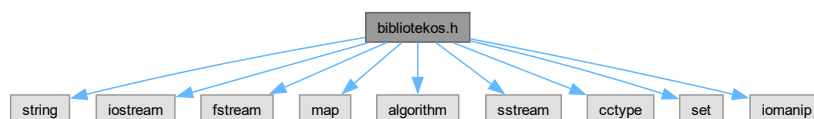
Chapter 4

File Documentation

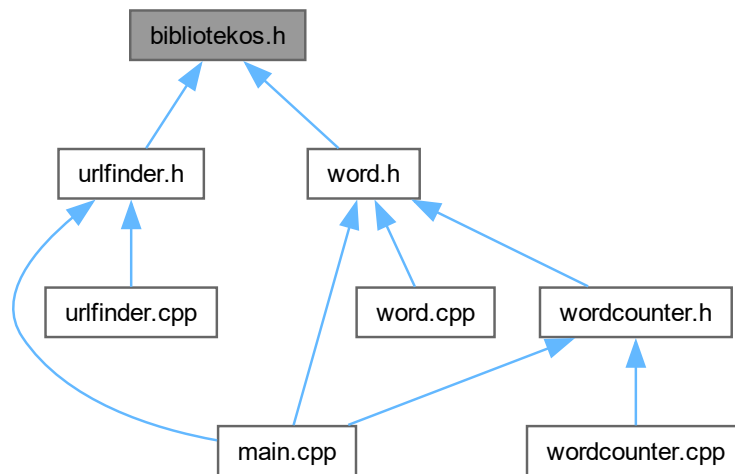
4.1 bibliotekos.h File Reference

```
#include <string>
#include <iostream>
#include <fstream>
#include <map>
#include <algorithm>
#include <sstream>
#include <cctype>
#include <set>
#include <iomanip>
```

Include dependency graph for bibliotekos.h:



This graph shows which files directly or indirectly include this file:



4.2 bibliotekos.h

[Go to the documentation of this file.](#)

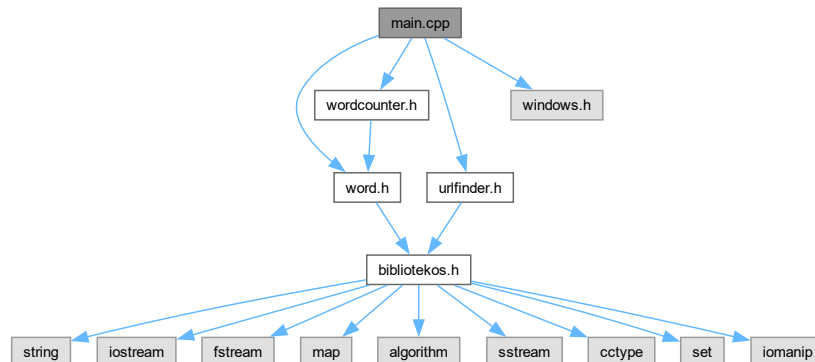
```
00001 #pragma once
00002 #include <string>
00003 #include <iostream>
00004 #include <fstream>
00005 #include <map>
00006 #include <algorithm>
00007 #include <sstream>
00008 #include <cctype>
00009 #include <set>
00010 #include <iomanip>
00011 using std::string;
00012 using std::map;
00013 using std::isalpha;
00014 using std::tolower;
00015 using std::ifstream;
00016 using std::cout;
00017 using std::endl;
00018 using std::ofstream;
00019 using std::ispunct;
00020 using std::set;
```

4.3 main.cpp File Reference

```
#include "word.h"
#include "wordcounter.h"
#include "urlfinder.h"
```

```
#include "windows.h"
```

Include dependency graph for main.cpp:



Functions

- int [main](#) ()

4.3.1 Function Documentation

4.3.1.1 main()

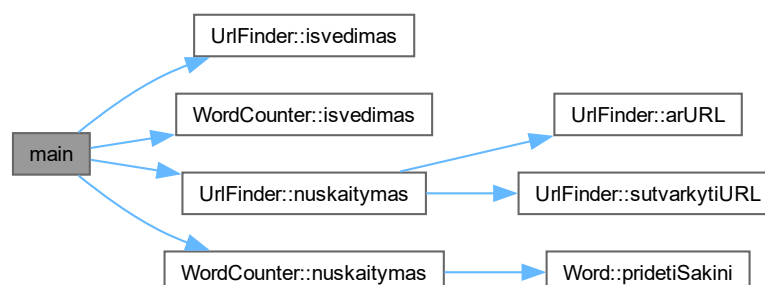
```
int main ()
```

Nustatome konsolės išvesties koduotę į UTF-8

Nustatome konsolės įvesties koduotę į UTF-8

Definition at line 6 of file [main.cpp](#).

Here is the call graph for this function:



4.4 main.cpp

[Go to the documentation of this file.](#)

```

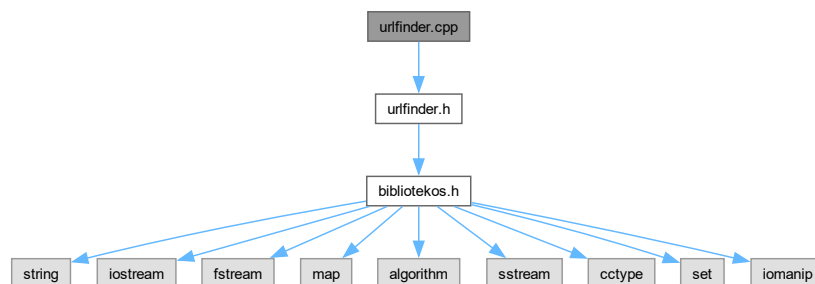
00001 #include "word.h"
00002 #include "wordcounter.h"
00003 #include "urlfinder.h"
00004
00005 #include "windows.h"
00006 int main()
00007 {
00008     SetConsoleOutputCP(CP_UTF8);
00009     SetConsoleCP(CP_UTF8);
00010
00011     WordCounter wc;
00012     UrlFinder uf;
00013
00014     cout<< " Ka jus norite padaryti? \n 1. Rasti zodzius ir ju skaiciu tekste \n 2. Rasti URL adresus
00015     tekste \n";
00016     int pasirinkimas;
00017     std::cin >> pasirinkimas;
00018     switch(pasirinkimas)
00019     {
00020         case 1:
00021             wc.nuskaitymas("tekstas.txt");
00022             wc.isvedimas("rezultatai.txt");
00023             break;
00024         case 2:
00025             uf.nuskaitymas("tekstas2.txt");
00026             uf.isvedimas("rezultatai2.txt");
00027             break;
00028         default:
00029             cout << "Neteisingas pasirinkimas. Programa baigiama." << endl;
00030     }
00031     cout<<"Programa baigta. Rezultatai issaugoti atitinkamuose failuose."<<endl;
00032     cout<<"Paspaukite Enter, kad uždarytumete programa..."<<endl;
00033     std::cin.ignore();
00034     std::cin.get();
00035     return 0;
00036 }

```

4.5 urlfinder.cpp File Reference

```
#include "urlfinder.h"
```

Include dependency graph for urlfinder.cpp:



4.6 urlfinder.cpp

[Go to the documentation of this file.](#)

```

00001 #include "urlfinder.h"
00002

```

```

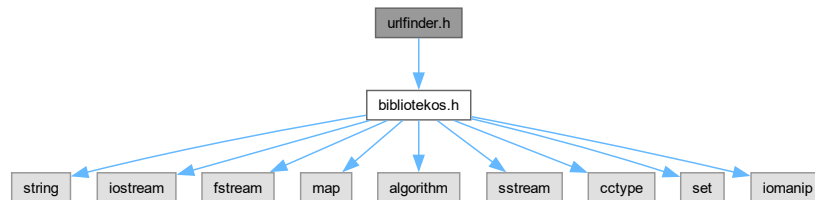
00003 bool UrlFinder::arURL(const string& zodis)
00004 {
00005     return zodis.rfind("https://", 0) == 0 ||
00006         zodis.rfind("http://", 0) == 0 ||
00007         zodis.rfind("www.", 0) == 0 ||
00008         zodis.find(".lt") != string::npos ||
00009         zodis.find(".com") != string::npos ||
00010         zodis.find(".net") != string::npos ||
00011         zodis.find(".org") != string::npos;
00012 }
00013 string UrlFinder::sutvarkytasURL(const string& zodis)
00014 {
00015     string sutvarkytasURL = zodis;
00017     while (!sutvarkytasURL.empty() &&
00018         (sutvarkytasURL.back() == '.' ||
00019         sutvarkytasURL.back() == ',' ||
00020         sutvarkytasURL.back() == '!' ||
00021         sutvarkytasURL.back() == '?' ||
00022         sutvarkytasURL.back() == ';' ||
00023         sutvarkytasURL.back() == ':' ||
00024         sutvarkytasURL.back() == '-' ||
00025         sutvarkytasURL.back() == '"' ||
00026         sutvarkytasURL.back() == '(' ||
00027         sutvarkytasURL.back() == ')' ||
00028         sutvarkytasURL.back() == '\\'))
00029     {
00030         sutvarkytasURL.pop_back();
00031     }
00032     while (!sutvarkytasURL.empty() &&
00033         (sutvarkytasURL.front() == '.' ||
00034         sutvarkytasURL.front() == ',' ||
00035         sutvarkytasURL.front() == '!' ||
00036         sutvarkytasURL.front() == '?' ||
00037         sutvarkytasURL.front() == ';' ||
00038         sutvarkytasURL.front() == ':' ||
00039         sutvarkytasURL.front() == '-' ||
00040         sutvarkytasURL.front() == '"' ||
00041         sutvarkytasURL.front() == '(' ||
00042         sutvarkytasURL.front() == ')' ||
00043         sutvarkytasURL.front() == '\\'))
00044     {
00045         sutvarkytasURL.erase(0,1);
00046     }
00047     return sutvarkytasURL;
00048 }
00049 void UrlFinder::nuskaitymas(const string& failas)
00050 {
00051     ifstream fr(failas);
00052     if(!fr.is_open())
00053     {
00054         cout << "Nepavyko atidaryti failo: " << failas << std::endl;
00055         return;
00056     }
00057     string eilute;
00058     while(fr >> eilute)
00059     {
00060         string url = sutvarkytasURL(eilute);
00061         if(arURL(url))
00062         {
00063             urls.insert(url);
00064         }
00065     }
00066 }
00067 void UrlFinder::isvedimas(const string& failas)
00068 {
00069     ofstream fd(failas);
00070     if(!fd.is_open())
00071     {cout << "Nepavyko atidaryti failo: " << failas << endl;
00072     return;
00073     }
00074     fd << "Rasti URL adresai:" << endl;
00075     fd<<"-----" << endl;
00076     for(const auto& url : urls)
00077     {
00078         fd << url << endl;
00079     }
00080 }

```

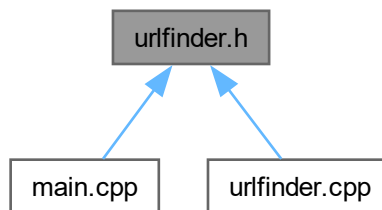
4.7 urlfinder.h File Reference

```
#include "bibliotekos.h"
```

Include dependency graph for urlfinder.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [UrlFinder](#)

4.8 urlfinder.h

[Go to the documentation of this file.](#)

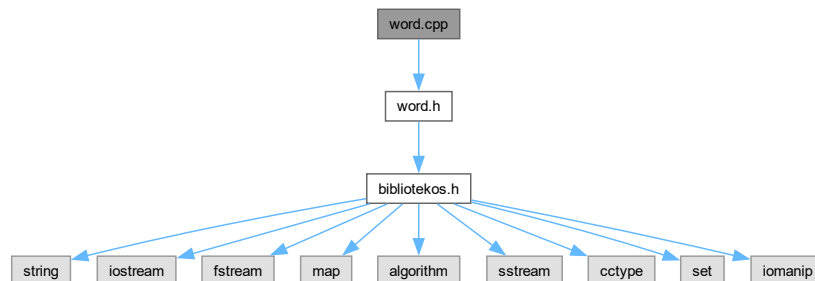
```

00001 #pragma once
00002 #include "bibliotekos.h"
00003
00004 class UrlFinder
00005 {
00006     private:
00007         set<string> urls;
00008
00009     public:
00010         bool arURL(const string& zodis);
00011         string sutvarkytiURL(const string& zodis);
00012         void nuskaitymas(const string& failas);
00013         void isvedimas(const string& failas);
00014
00015 };
00016 
```


4.9 word.cpp File Reference

```
#include "word.h"
```

Include dependency graph for word.cpp:



Functions

- `std::ostream & operator<< (std::ostream &os, const Word &zodis)`
perdengtas išvedimo operatorius

4.9.1 Function Documentation

4.9.1.1 operator<<()

```
std::ostream & operator<< (
    std::ostream & os,
    const Word & zodis)
```

perdengtas išvedimo operatorius

Definition at line 65 of file [word.cpp](#).

4.10 word.cpp

[Go to the documentation of this file.](#)

```

00001
00002 #include "word.h"
00003
00004 Word::Word()
00005 {
00006     word = "";
00007     count = 0;
00008 }
00009
00010 Word::Word(const string& word, int count)
00011 {
00012     this->word = word;
00013     this->count = count;
00014 }
00015 Word::Word(const string& word, int count, int sakinyys)
00016 {
```

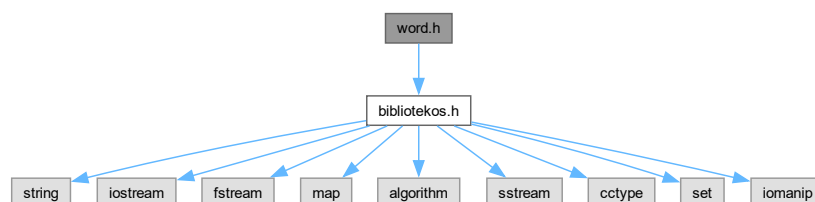
```

00017     this->word = word;
00018     this->count = count;
00019     this->sakiniai.insert(sakinys);
00020 }
00021 Word::Word(const Word&zodis)
00022 {
00023     word = zodis.word;
00024     count = zodis.count;
00025     sakiniai = zodis.sakiniai;
00026 }
00027 Word& Word::operator=(const Word& zodis)
00028 {
00029     if(this == &zodis)
00030         return *this;
00031     word = zodis.word;
00032     count = zodis.count;
00033     sakiniai = zodis.sakiniai;
00034 }
00035     return *this;
00036 }
00037 Word::Word(Word&& zodis)
00038 {
00039     word = std::move(zodis.word);
00040     count = zodis.count;
00041     sakiniai = std::move(zodis.sakiniai);
00042     zodis.word.clear();
00043     zodis.count = 0;
00044     zodis.sakiniai.clear();
00045 }
00046 Word& Word::operator=(Word&& zodis)
00047 {
00048     if(this == &zodis)
00049         return *this;
00050     word = std::move(zodis.word);
00051     count = zodis.count;
00052     sakiniai = std::move(zodis.sakiniai);
00053     zodis.word.clear();
00054     zodis.count = 0;
00055     zodis.sakiniai.clear();
00056 }
00057     return *this;
00058 }
00059 Word::~Word()
00060 {
00061     word.clear();
00062     count = 0;
00063     sakiniai.clear();
00064 }
00065 std::ostream& operator<<(std::ostream& os, const Word& zodis)
00066 {
00067     os << std::left << std::setw(25) << zodis.getWord() << std::left << std::setw(10) << zodis.getCount();
00068     for (const auto& sakinys : zodis.getSakiniai())
00069     {
00070         os << sakinys << " ";
00071     }
00072     os << endl;
00073     return os;
00074 }

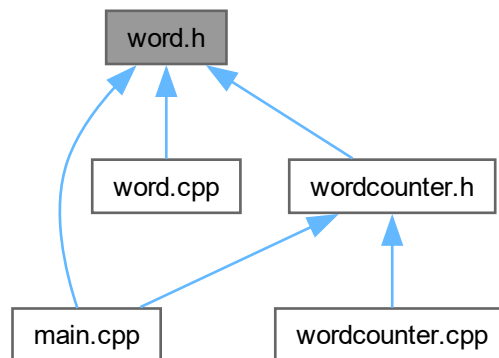
```

4.11 word.h File Reference

#include "bibliotekos.h"
 Include dependency graph for word.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [Word](#)

4.12 word.h

[Go to the documentation of this file.](#)

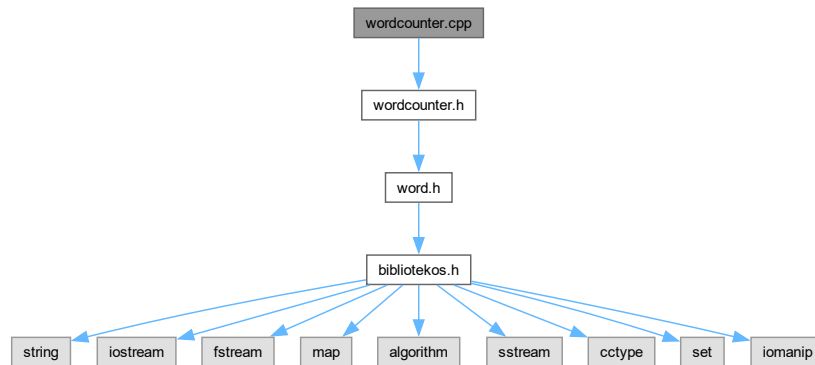
```

00001
00002 #pragma once
00003 #include "bibliotekos.h"
00004
00005 class Word
00006 {
00007     private:
00008         string word;
00009         int count;
00010         std::set<int> sakiniai;
00011     public:
00012         Word();
00013         Word(const string& word, int count);
00014         Word(const string& word, int count, int sakiny);
00015         Word(const Word& zodis);
00016         Word& operator=(const Word& zodis);
00017         Word(Word&& zodis);
00018         Word& operator=(Word&& zodis);
00019         ~Word();
00020         const string& getWord()const {return word;};
00021         int getCount()const {return count;};
00022         void setWord(const string& word) {this->word = word;};
00023         void setCount(int count) {this->count = count;};
00024
00025         void didintiCount() {this->count++;};
00026         void pridetiSakini(int sakinioNumeris) {sakiniai.insert(sakinioNumeris);};
00027         const std::set<int>& getSakiniai() const {return sakiniai;};
00028         friend std::ostream& operator<<(std::ostream& os, const Word& zodis);
00029 };
  
```

4.13 wordcounter.cpp File Reference

```
#include "wordcounter.h"
```

Include dependency graph for wordcounter.cpp:



4.14 wordcounter.cpp

[Go to the documentation of this file.](#)

```

00001 #include "wordcounter.h"
00002 string WordCounter::arZodis(const string& zodis)
00003 {
00004     string rezultatas;
00005     for (size_t i = 0; i < zodis.size(); )
00006     {
00007         unsigned char c = zodis[i];
00008
00009         if (c >= 0x80)
00010         {
00011             int baitai = 1;
00012             if (c >= 0xF0) baitai = 4;
00013             else if (c >= 0xE0) baitai = 3;
00014             else if (c >= 0xC0) baitai = 2;
00015
00016             string simbolis = zodis.substr(i, baitai);
00017
00018             if (c == 0xC3 || c == 0xC5) {
00019                 for (char ch : simbolis) rezultatas += ch;
00020             }
00021             i += baitai;
00022             continue;
00023         }
00024
00025         if (c == ',' || c == '/' || c == '!' || c == '?' ||
00026             c == ';' || c == ':' || c == '-' || c == '"' ||
00027             c == '(' || c == ')') {
00028             i++;
00029             continue;
00030         }
00031
00032         rezultatas += (char)tolower(c);
00033         i++;
00034     }
00035     return rezultatas;
00036 }
00037
00038 void WordCounter::nuskaitymas(const string& failas)
00039 {
00040     ifstream fr(failas);
00041     if (!fr.is_open())
00042     {
00043         cout << "Nepavyko atidaryti failo: " << failas << std::endl;
00044         return;
00045     }
00046 }

```

```

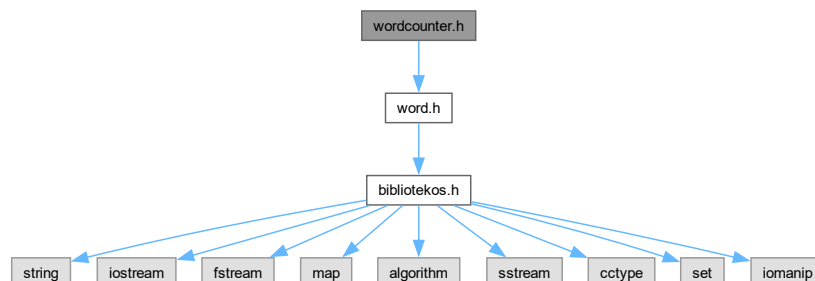
00053     string eilute;
00054     int sakinioNumeris = 1;
00055     while(fr » eilute)
00056     {
00057         bool sakinioPabaiga = false;
00058         if(eilute.find('.') != string::npos || eilute.find('!' ) != string::npos ||
eilute.find('?') != string::npos)
00059         {
00060             sakinioPabaiga = true;
00061         }
00062
00063         string tuscia = arZodis(eilute);
00064         if (tuscia.empty()) {if(sakinioPabaiga) sakinioNumeris++; continue;}
00065
00066         if (zodziai.find(tuscia) == zodziai.end())
00067         {
00068             Word naujasZodis(tuscia, 1);
00069             naujasZodis.pridetiSakini(sakinioNumeris);
00070             zodziai.insert({tuscia, naujasZodis});
00071         } else {
00072             zodziai[tuscia].setWord(tuscia);
00073             zodziai[tuscia].didintiCount();
00074             zodziai[tuscia].pridetiSakini(sakinioNumeris);
00075         }
00076         if(sakinioPabaiga) sakinioNumeris++;
00077     }
00078     fr.close();
00079 }
00080 }
00081 void WordCounter::isvedimas(const string& failas)
00082 {
00083     ofstream fd(failas);
00084     if(!fd.is_open())
00085     {
00086         cout << "Nepavyko atidaryti failo: " << failas << std::endl;
00087         return;
00088     }
00089     fd << std::left << std::setw(25) << "Zodis" << std::setw(10) << "Kiekis" << "Sakiniai" << endl;
00090     for(const auto& pora : zodziai)
00091     {
00092         if(pora.second.getCount() > 1)
00093         {
00094             fd << pora.second;
00095         }
00096     }
00097     fd.close();
00098 }

```

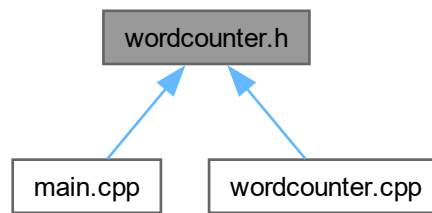
4.15 wordcounter.h File Reference

```
#include "word.h"
```

Include dependency graph for wordcounter.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [WordCounter](#)

4.16 wordcounter.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002 #include "word.h"
00003
00004 class WordCounter
00005 {
00006     private:
00007         map<string, Word> zodziai;
00008         string arZodis(const string& zodis);
00009
00010     public:
00011         void nuskaitymas(const string& failas);
00012         void isvedimas(const string& failas);
00013 };
```

Index

- ~Word
 - Word, [9](#)
- arURL
 - UrlFinder, [5](#)
- bibliotekos.h, [15](#)
- didintiCount
 - Word, [10](#)
- getCount
 - Word, [10](#)
- getSakiniai
 - Word, [10](#)
- getWord
 - Word, [10](#)
- isvedimas
 - UrlFinder, [5](#)
 - WordCounter, [13](#)
- main
 - main.cpp, [17](#)
- main.cpp, [16](#)
 - main, [17](#)
- nuskaitymas
 - UrlFinder, [6](#)
 - WordCounter, [13](#)
- operator<<
 - Word, [12](#)
 - word.cpp, [21](#)
- operator=
 - Word, [11](#)
- pridetiSakini
 - Word, [11](#)
- setCount
 - Word, [12](#)
- setWord
 - Word, [12](#)
- sutvarkytiURL
 - UrlFinder, [6](#)
- UrlFinder, [5](#)
 - arURL, [5](#)
 - isvedimas, [5](#)
 - nuskaitymas, [6](#)
 - sutvarkytiURL, [6](#)
- urlfinder.cpp, [18](#)
- urlfinder.h, [20](#)
- Word, [7](#)
 - ~Word, [9](#)
 - didintiCount, [10](#)
 - getCount, [10](#)
 - getSakiniai, [10](#)
 - getWord, [10](#)
 - operator<<, [12](#)
 - operator=, [11](#)
 - pridetiSakini, [11](#)
 - setCount, [12](#)
 - setWord, [12](#)
 - Word, [8, 9](#)
- word.cpp, [21](#)
 - operator<<, [21](#)
- word.h, [22](#)
- WordCounter, [13](#)
 - isvedimas, [13](#)
 - nuskaitymas, [13](#)
- wordcounter.cpp, [24](#)
- wordcounter.h, [25](#)